Justificación de la Estructura de Datos y Análisis de Complejidad

Proyecto Nro. 1 — SparseSpreadsheet
CS2100: Algoritmos y Estructuras de Datos

---

1. Justificación de la estructura de datos

Se eligió una **matriz dispersa con listas enlazadas cruzadas**.
Una hoja de cálculo típica (ej. 1000×26 = 26 000 celdas) mantiene ocupada una fracción mínima de sus celdas. Un arreglo denso reserva O(F×C) memoria, desperdiciando espacio y complicando operaciones masivas.

La lista enlazada cruzada **solo almacena celdas con datos**. Cada nodo contiene su valor y dos punteros (`next_row`, `next_col`). Los vectores de cabeceras (`rows[]`, `cols[]`) crecen dinámicamente y permiten recorrer una fila/columna en tiempo proporcional a sus elementos ocupados.

Frente a otras alternativas:

- Matriz densa: O(F×C) memoria fija; eliminar fila mueve toda la estructura.
- `std::unordered_map`: iteración ordenada ineficiente, mala para agregaciones.
- `std::map`: sobrecarga de memoria sin recorridos bidimensionales nativos.
- Lista enlazada simple: recorridos verticales O(k_total).

---

2. Análisis de complejidad espacial

- **Espacio:** O(k + F + C) ≈ O(k), con k = celdas ocupadas, F = filas activas, C = columnas activas.
- Solo existen nodos para celdas con datos; las cabeceras vacías son punteros nulos.
- **Ejemplo (1000×26, 1%):** denso → ~ 26 000 entradas (~ 208 KB); lista cruzada → 260 nodos (~5-10 KB).

---

3. Análisis de complejidad temporal

- *k_row*: celdas no vacías en la fila objetivo.
- *k_col*: celdas no vacías en la columna objetivo.
- *F*: filas activas totales.
- *C*: columnas activas totales.
- *n*: longitud de la fórmula.

Operación	Complejidad (peor caso)	Explicación
`insertarCelda`	O(F + C)	Redimensiona vectores y busca posición en fila y columna.
`consultarCelda`	O(k_row)	Accede por índice a la fila y recorre sus nodos.
`modificarCelda`	O(k_row)	Ídem a `consultarCelda`.
`eliminarCelda`	O(k_row + k_col)	Desenlaza el nodo de su fila y de su columna.
`eliminarFila`	O(k_row × C)	Desenlaza cada nodo de su respectiva columna.
`eliminarColumna`	O(k_col × F)	Análogo a `eliminarFila`.
`eliminarRango`	O((f2−f1)×(c2−c1))	Invoca `eliminarCelda` por cada celda del rectángulo.
`sumaFila`	O(k_row)	Recorre los nodos de la fila y acumula valores numéricos.
`sumaColumna`	O(k_col)	Recorre los nodos de la columna.
`sumaRango`	O(F + k_total)	Itera sobre las filas del rango y recorre los nodos presentes.
`promedioFila`	O(k_row)	Cuenta y suma los valores numéricos de la fila.
`promedioColumna`	O(k_col)	Análogo sobre una columna.
`promedioRango`	O(F + k_total)	Recorre el rectángulo contando y sumando.
`maxRango`	O(F + k_total)	Determina el valor numérico máximo en el rango.
		Determina el valor numérico

`minRango`	O(F + k_total)	Determina el valor numérico mínimo en el rango.
`FormulaEvaluator::evaluar`	O(n)	Parser de precedencia que procesa la expresión linealmente.